

EE-2003

Computer Organization and Assembly Language

Week 15 – Lecture 29

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

Outline

- 1 Quick Review
- 2 Finding String Length
- 3 String Length Routine
- 4 Number Printing: Extracting Digits
- 5 Understanding SCASB (String Scan Byte)
- 6 Visualization: How SCASB Finds String Length

Quick Review: What was in the Last Week?

- REP, STOSB, STOSW Instructions

Program Overview

- This program demonstrates three major concepts in assembly:
 - **Clearing the screen** using `clrscr`
 - **Finding the length of a string** using `strlen`
 - **Printing a number** using `printnum`
- Focus on 8088 instructions:
 - `REP`, `STOSW`, `SCASB`
 - `DIV` for digit extraction
 - Stack-based parameter passing
 - ASCII conversion for display
- Final outcome:
 - Clear screen
 - Compute `strlen("hello world")`
 - Display length (11) on screen

Screen Clearing: `clrscr`

Purpose: Fill the entire video memory with blank characters.

- `mov ax, 0xB800, mov es, ax` → ES points to video memory.
- `xor di, di` → DI = 0 (top-left screen position).
- `mov ax, 0x0720` → Character = space (20h), attribute = 07h.
- `mov cx, 2000` → $80 \times 25 = 2000$ character cells.
- **`cld + rep stosw`:**
 - STOSW stores AX at ES:DI
 - DI auto-increments by 2 (word)
 - REP repeats 2000 times
- Entire screen is cleared.

String Length Routine: strlen

Purpose: Count characters until NULL (00h).

- `les di, [bp+4]` → Load ES:DI with starting address of string.
- `mov cx, 0xFFFF` → Maximum possible search length.
- `xor al, al` → $AL = 0$ (value to search for).
- `repne scasb`:
 - Compare byte at ES:DI to AL (00h)
 - Continue while NOT equal
 - DI increments each time
 - CX decrements each time
- **Length** = $0xFFFF - CX - 1$ → Minus 1 to exclude the NULL.

Number Printing: Extracting Digits

Goal: Convert binary number into ASCII digits.

- $AX = \text{number}$, $BX = 10$ (base 10), $CX = \text{digit count}$
- Loop:
 - `mov dx, 0` — required for 16-bit DIV
 - `div bx` → AX/BX
 - Quotient → AX
 - Remainder → DL
 - Remainder is digit → convert using **`add dl, 0x30`**
 - Push DL on stack
 - Increase digit count
- Stop when $AX = 0$ (no more digits).

Number Printing: Displaying Digits

Goal: Pop digits from stack and print left-to-right.

- `mov ax, 0xB800, mov es, ax`
- `DI = 0` → top-left screen location
- Loop (CX times):
 - `pop dx` → ASCII digit
 - `mov dh, 0x07` → attribute (white on black)
 - `mov [es:di], dx` → prints digit
 - `add di, 2` → next screen cell

Understanding SCASB (String Scan Byte)

SCASB compares AL with the byte at ES:DI and then moves DI forward.

- AL = value to search (in our case: 00h)
- ES:DI = current character in the string
- SCASB steps:
 - Compare AL with [ES:DI]
 - $DI = DI + 1$
 - $CX = CX - 1$
 - Flags updated ($ZF = 1$ if match)
- REPNE SCASB: Repeat until NOT equal ($ZF = 1$ stop).

Used to detect the NULL terminator in strings.

Visualization: How SCASB Finds String Length

String: 'h' 'e' 'l' 'l' 'o' ' ' 'w' 'o' 'r' 'l' 'd' 00h

h	e	l	l	o	_	w	o	r	l	d	00
DI ₀	DI ₁	DI ₂	DI ₃	DI ₄	DI ₅	DI ₆	DI ₇	DI ₈	DI ₉	DI ₁₀	DI ₁₁

- AL = 00h (search value)
- SCASB compares AL with each byte
- DI increments after each comparison
- Stops when it reaches the first 00h
- Number of comparisons = string length + 1

Example: SCASB stops at DI = 11 → length = 11 characters.

Thank You!

Any Questions?

- Today we learned how **8088 string instructions** automate complex tasks like clearing the screen and finding string lengths.
- The **CLRSCR routine** showed how REP STOSW can fill all 2000 screen cells efficiently.
- The **number-printing routine** demonstrated digit extraction using DIV, converting values to ASCII, and printing them using stack reversal.
- The **STRLEN routine** used LES, SCASB, and REPNE to scan memory until the NULL byte and compute length automatically.
- We saw how registers like **ES:DI**, **DS:SI**, **CX**, and the **DF flag** control direction and memory traversal.

“With string instructions, the CPU itself performs the loop—making scanning, filling, and searching far faster and simpler.”